TECNICATURA UNIVERSITARIA EN DESARROLLO WEB · FCAD · UNER

Unidad 4
APIs del dispositivo

Cámara · Galería · Ubicación · Mapas · Sensores · Hápticos · Archivos

Desarrollo para Móviles · 2026 — Segundo cuatrimestre

Expo SDK 57 · React Native 0.86 · React 19.2

UNIDAD 4

Qué vamos a ver

Objetivos

- Sacar y elegir imágenes desde la app.
- Ubicar al usuario y mostrarlo en un mapa.
- Leer sensores y devolver respuesta táctil.
- Guardar y leer archivos con la API nueva.
- Reproducir video y audio, y grabar con el micrófono.

Partes

- Cámara y galería
- Ubicación y mapas
- Sensores y hápticos
- Sistema de archivos
- Multimedia

> Todo lo de hoy asume el deck anterior: `npx expo install`, config plugin y permiso pedido en tiempo de ejecución.

UNIDAD 4 · PARTE 1

Cámara y galería

Dos librerías distintas para dos problemas parecidos.

Sacar la foto vs. elegir la foto

`expo-camera`

- Te da un **componente**: la vista previa vive dentro de tu pantalla.
- Controlás zoom, linterna, flash, cámara frontal o trasera.
- Escanea códigos QR y de barras.
- Interfaz propia: la diseñás vos.

`expo-image-picker`

- Te da una **función**: abre la pantalla del sistema.
- El usuario ya conoce esa interfaz.
- Permite recortar y elegir varias.
- También puede abrir la cámara del sistema.

> Para el TP integrador, empezá por `expo-image-picker`: resuelve el 90% de los casos con diez líneas. Pasá a `expo-camera` solo si necesitás la vista previa dentro de tu pantalla o escanear códigos.

La vista previa

```tsx
import { CameraView, CameraType, useCameraPermissions } from 'expo-camera';
import { useState } from 'react';

export default function Camara() {
  const [lado, setLado] = useState<CameraType>('back');
  const [permiso, pedirPermiso] = useCameraPermissions();

  // …las tres ramas del permiso, como en el deck anterior…

  return (
    <CameraView style={{ flex: 1 }} facing={lado} enableTorch={false}>
      <Pressable onPress={() => setLado(l => (l === 'back' ? 'front' : 'back'))}>
        <Text>Dar vuelta</Text>
      </Pressable>
    </CameraView>
  );
}
```

> `<CameraView>` acepta hijos: los botones se dibujan **encima** de la vista previa, como en cualquier `View`.

Sacar la foto

```tsx
import { CameraView } from 'expo-camera';
import { useRef, useState } from 'react';

const camara = useRef<CameraView>(null);
const [foto, setFoto] = useState<string | null>(null);

async function disparar() {
  const resultado = await camara.current?.takePictureAsync({ quality: 0.7 });
  setFoto(resultado?.uri ?? null);   // file:///…/Caches/ExponentExperienceData/…
}

<CameraView ref={camara} style={{ flex: 1 }} />
```

Qué devuelve

`uri`, `width`, `height`. Con `base64: true` también el contenido, pero pesa un tercio más.

Dónde queda

En la **caché** de la app. El sistema la puede borrar. Si la foto tiene que sobrevivir, hay que copiarla — lo vemos en la Parte 4.

Escanear un código

```
<CameraView
  style={{ flex: 1 }}
  barcodeScannerSettings={{ barcodeTypes: ['qr', 'ean13'] }}
  onBarcodeScanned={escaneado ? undefined : ({ type, data }) => {
    setEscaneado(true);          // sin esto se dispara decenas de veces por segundo
    router.push(`/producto/${data}`);
  }}
/>
```

`onBarcodeScanned` se llama en **cada cuadro** donde el código sea visible. Sin una bandera de estado vas a navegar treinta veces a la misma pantalla.

Idea para el TP: inventario, entradas a un evento, menú de un restaurante, control de asistencia. Es la funcionalidad que más impresiona por menos código.

Elegir de la galería

```
import * as ImagePicker from 'expo-image-picker';

async function elegirImagen() {
  const resultado = await ImagePicker.launchImageLibraryAsync({
    mediaTypes: ['images'],        // también 'videos' y 'livePhotos'
    allowsEditing: true,
    aspect: [4, 3],
    quality: 0.8,
  });

  if (resultado.canceled) return;

  const imagen = resultado.assets[0];
  setUri(imagen.uri);
}
```

`mediaTypes` ahora es un **array de strings**: si encontrás `MediaTypeOptions.Images`, ese tutorial es viejo DEPRECADO. Con `allowsMultipleSelection` vienen varias en `assets`, y `canceled` va con una sola l.

Qué te devuelve, y cómo lo mostrás

EL ASSET

```
{
  uri: 'file:///…/IMG_0042.jpg',
  width: 3024,
  height: 4032,
  fileSize: 2_481_920,
  mimeType: 'image/jpeg',
  fileName: 'IMG_0042.jpg',
  exif: { … }
}
```

MOSTRARLO

```
import { Image } from 'expo-image';

<Image
  source={{ uri }}
  style={{ width: '100%', aspectRatio: 3 / 4 }}
  contentFit="cover"
  transition={200}
/>
```

Usá `expo-image` y no el `Image` de React Native: caché en disco, placeholder y transición al aparecer. Y ojo con `fileSize`: una foto de teléfono son 2 a 5 MB, así que antes de subirla redimensionala con `expo-image-manipulator`.

UNIDAD 4 · PARTE 2

Ubicación y mapas

Dónde está el usuario, y cómo se lo mostrás.

Dos permisos, no uno

`requestForegroundPermissionsAsync()`

«Mientras uso la app». Es el que vas a usar en la materia. El usuario ve un cartel normal.

`requestBackgroundPermissionsAsync()`

«Siempre», incluso con la app cerrada. Requiere justificación ante las dos tiendas y en Android hay que pedir el de primer plano primero.

> No pidas el de segundo plano «por las dudas». Es el permiso que más rechazos de publicación genera, y para un TP integrador casi nunca hace falta.

Obtener la posición

```
import * as Location from 'expo-location';

async function ubicar() {
  const { status } = await Location.requestForegroundPermissionsAsync();
  if (status !== 'granted') return null;

  const ultima = await Location.getLastKnownPositionAsync();   // rápida, puede ser vieja
  if (ultima) mostrar(ultima);

  const actual = await Location.getCurrentPositionAsync({      // precisa, tarda
    accuracy: Location.Accuracy.Balanced,
  });
  mostrar(actual);
}
```

> Ese doble pedido es el patrón que usan las apps buenas: mostrás algo **ya** y lo corregís cuando llega la lectura fina. Sin eso el usuario mira una pantalla vacía cinco segundos.

Seguir la posición

```
useEffect(() => {
  let suscripcion: Location.LocationSubscription;

  (async () => {
    const { status } = await Location.requestForegroundPermissionsAsync();
    if (status !== 'granted') return;

    suscripcion = await Location.watchPositionAsync(
      { accuracy: Location.Accuracy.High, distanceInterval: 10 },  // metros
      (posicion) => setCoords(posicion.coords),
    );
  })();

  return () => suscripcion?.remove();   // ← sin esto el GPS sigue prendido
}, []);
```

El `return` del `useEffect` no es opcional. Una suscripción de GPS que queda viva después de salir de la pantalla vacía la batería y es el bug que más aparece corregido en las entregas.

De coordenadas a dirección y al revés

COORDENADAS → DIRECCIÓN

```
const [lugar] = await Location.reverseGeocodeAsync({
  latitude: -31.7333,
  longitude: -60.5297,
});

// { city: 'Paraná', region: 'Entre Ríos',
//   street: 'Urquiza', country: 'Argentina' }
```

DIRECCIÓN → COORDENADAS

```
const [punto] = await Location.geocodeAsync(
  'Urquiza 552, Paraná, Entre Ríos'
);

// { latitude: -31.7333, longitude: -60.5297 }
```

Los nombres se confunden todo el tiempo. **Geocodificar** es convertir un texto en coordenadas; **reverse** es el camino inverso. Los dos devuelven un **array**, que puede venir vacío.

En Android hay que tener el permiso de ubicación concedido antes de geocodificar, aunque no estés leyendo la posición del usuario.

El mapa

REACT-NATIVE-MAPS LO QUE USAMOS

```
import MapView, { Marker, Circle } from 'react-native-maps';

<MapView
  style={{ flex: 1 }}
  initialRegion={{
    latitude: -31.7333, longitude: -60.5297,
    latitudeDelta: 0.05, longitudeDelta: 0.05,
  }}>
  <Marker
    coordinate={{ latitude: -31.7333, longitude: -60.5297 }}
    title="FCAD" description="Facultad de Ciencias de la Administración"
  />
  <Circle
    center={{ latitude: -31.7333, longitude: -60.5297 }}
    radius={500}
  />
</MapView>
```

QUÉ PODÉS DIBUJAR ENCIMA

- `Marker` — un pin. Acepta un componente propio como hijo.
- `Polyline` — un recorrido.
- `Polygon` — un área.
- `Circle` — centro y radio en metros.

Todos van como **hijos directos** de `<MapView>`.

`latitudeDelta` es el zoom: cuántos grados entran en la pantalla.

> Viene incluido en Expo Go (`1.27.2` en SDK 57): se prueba sin configurar nada.

El mapa en producción

En desarrollo

Nada. Expo Go trae la librería y usa Apple Maps en iOS y Google Maps en Android, sin credenciales.

En la build de la tienda

Google Maps exige una **clave de API** propia, restringida por huella SHA-1 y por identificador de paquete.

```
{
  "expo": {
    "plugins": [
      ["react-native-maps", { "androidGoogleMapsApiKey": "process.env.GOOGLE_MAPS_KEY" }]
    ]
  }
}
```

> La clave **no va escrita en el repositorio**: variable de entorno y, en EAS, un secret. Lo retomamos en la Unidad 7.

`expo-maps`: hacia dónde va el SDK

Qué propone

Un módulo propio de Expo, con un componente por plataforma en vez de una abstracción común.

```
import { AppleMaps, GoogleMaps } from 'expo-maps';

Platform.OS === 'ios'
  ? <AppleMaps.View style={{ flex: 1 }} />
  : <GoogleMaps.View style={{ flex: 1 }} />
```

Por qué todavía no lo usamos ALPHA

- La propia documentación avisa que va a tener **cambios que rompen**, seguido.
- **No está en Expo Go**: hace falta development build.
- Google Maps solo en Android, Apple Maps solo en iOS. No hay un componente único.

Decisión de la cursada: se enseña `react-native-maps` y se muestra `expo-maps` para que sepan que existe. Si tu grupo quiere usarlo en el TP, se puede — pero asumiendo que puede romperse en una actualización.

UNIDAD 4 · PARTE 3

Sensores y hápticos

Leer el mundo físico y contestarle.

Lo que trae `expo-sensors`

Acelerómetro
Aceleración en g sobre x, y, z. Inclinación, sacudidas, pasos.

Giroscopio
Velocidad de rotación en rad/s. Orientación del dispositivo.

Magnetómetro
Campo magnético en µT. Es lo que hace funcionar una brújula.

Barómetro
Presión en hPa. Altitud aproximada, clima. No todos lo tienen.

LightSensor
Luz ambiente en lux. **Solo Android.**

Podómetro
Conteo de pasos, en vivo o histórico.

DeviceMotion
Los tres primeros combinados y ya procesados. El más práctico.

Todos
Misma API: suscribirse, fijar frecuencia, desuscribirse.

En el simulador de iOS los sensores **no existen**. Esta parte se prueba sí o sí en un teléfono real.

El patrón, siempre igual

```tsx
import { useEffect, useState } from 'react';
import { Accelerometer } from 'expo-sensors';

useEffect(() => {
  let suscripcion: { remove: () => void };

  (async () => {
    if (!(await Accelerometer.isAvailableAsync())) return;   // 1 · ¿existe?

    Accelerometer.setUpdateInterval(100);                    // 2 · cada 100 ms
    suscripcion = Accelerometer.addListener(setDatos);       // 3 · escuchar
  })();

  return () => suscripcion?.remove();                        // 4 · soltar
}, []);
```

Si encontrás `activate()`, `deactivate()` o `removeAllListeners()` en un apunte, está viejo: los dos primeros no existen y el tercero está **DEPRECADO** en favor de `subscription.remove()`.

Un ejemplo entero: detectar una sacudida

```
import { Accelerometer } from 'expo-sensors';
import * as Haptics from 'expo-haptics';

const UMBRAL = 1.8;   // en g. En reposo el módulo del vector da ≈ 1.

useEffect(() => {
  Accelerometer.setUpdateInterval(100);
  const sub = Accelerometer.addListener(({ x, y, z }) => {
    if (Math.sqrt(x * x + y * y + z * z) > UMBRAL) {
      Haptics.notificationAsync(Haptics.NotificationFeedbackType.Success);
      sortearPregunta();
    }
  });
  return () => sub.remove();
}, []);
```

> El módulo del vector no depende de cómo esté agarrado el teléfono: sirve igual en cualquier orientación. Una sacudida firme pasa de 1,8.

Hápticos: tres tipos de respuesta

`selectionAsync()`

Cambió una selección. Elegir un ítem, mover un slider, pasar de pestaña.

El más sutil.

`notificationAsync(tipo)`

`Success`, `Warning`, `Error`.

Resultado de una acción: se guardó, falló el pago.

`impactAsync(estilo)`

`Light`, `Medium`, `Heavy`, `Rigid`, `Soft`. Un choque físico simulado.

En Android la documentación recomienda `performAndroidHapticsAsync(AndroidHaptics.Confirm)` antes que `impactAsync`: usa el motor háptico del sistema y no necesita el permiso `VIBRATE`.

En iOS no pasa nada con el modo de bajo consumo activo, con el Taptic Engine apagado o con la cámara en uso. **Nunca uses la vibración como única señal.**

UNIDAD 4 · PARTE 4

Sistema de archivos

API nueva desde el SDK 54. La anterior ya no funciona.

Esto cambió, y cambió fuerte

Las funciones sueltas — `writeAsStringAsync`, `readAsStringAsync`, `getInfoAsync`… — **tiran error en tiempo de ejecución**. No están deprecadas: no andan.

ANTES SDK 53 Y ANTERIORES

```
import * as FileSystem from 'expo-file-system';

const uri = FileSystem.documentDirectory + 'notas.txt';
await FileSystem.writeAsStringAsync(uri, texto);
const leido = await FileSystem.readAsStringAsync(uri);
```

AHORA SDK 54 EN ADELANTE

```
import { File, Paths } from 'expo-file-system';

const archivo = new File(Paths.document, 'notas.txt');
archivo.create();
archivo.write(texto);
const leido = archivo.textSync();
```

Si tenés código viejo que no querés reescribir hoy, existe la puerta de servicio: `import * as FileSystem from 'expo-file-system/legacy'`. Sirve para migrar de a poco, no para arrancar un proyecto nuevo.

La tabla de traducción

API vieja

- `documentDirectory`
- `cacheDirectory`
- `writeAsStringAsync()`
- `readAsStringAsync()`
- `deleteAsync()` · `moveAsync()` · `copyAsync()`
- `makeDirectoryAsync()`
- `readDirectoryAsync()`
- `getInfoAsync()`
- `downloadAsync()`

API nueva

- `Paths.document`
- `Paths.cache`
- `archivo.write()`
- `archivo.text()` · `textSync()`
- `archivo.delete()` · `move()` · `copy()`
- `new Directory(…).create()`
- `directorio.list()`
- `archivo.exists` · `archivo.info`
- `File.downloadFileAsync()`

El cambio de fondo: de **funciones sobre strings** a **objetos con métodos**. Un `File` sabe si existe, cuánto pesa y cómo leerse; ya no arrastrás la URI a todos lados.

Dónde guardar cada cosa

`Paths.document`

Permanente. Sobrevive a reinicios y actualizaciones. Se borra solo al desinstalar.

Notas del usuario, borradores, la foto que subió.

`Paths.cache`

Temporal. El sistema lo puede vaciar cuando le falta espacio, sin avisarte.

Miniaturas, descargas en curso, la salida de la cámara.

Diagnóstico

`Paths.availableDiskSpace` y `Paths.totalDiskSpace` reemplazan a las viejas funciones de espacio libre.

En iOS el sistema **revisa** qué guardás en documentos. Poner ahí archivos que se pueden volver a descargar es motivo de rechazo en la App Store. Descargas → caché.

Leer, escribir, recorrer

```
import { File, Directory, Paths } from 'expo-file-system';

const carpeta = new Directory(Paths.document, 'notas');     // crear
if (!carpeta.exists) carpeta.create();

const archivo = new File(carpeta, 'clase-09.md');
archivo.create({ overwrite: true });
archivo.write('# Unidad 4');

console.log(archivo.textSync());                            // leer, sincrónico
console.log(await archivo.text());                          // …o asincrónico

for (const item of carpeta.list()) {                        // recorrer
  if (item instanceof File) console.log(item.name, item.size);
}
```

> Las versiones sincrónicas son nuevas y son cómodas: para un archivo de configuración de 2 KB no tiene sentido un `await`. Para algo grande, seguí usando la asincrónica o vas a trabar la interfaz.

Descargar con barra de progreso

```
import { File, Directory, Paths } from 'expo-file-system';

// Simple: se resuelve cuando terminó
const destino = new Directory(Paths.cache, 'pdfs');
if (!destino.exists) destino.create();
const bajado = await File.downloadFileAsync('https://…/apunte.pdf', destino);

// Con progreso: lo que necesitás para mostrar una barra
const salida = new File(Paths.cache, 'video.mp4');
const tarea = File.createDownloadTask('https://…/video.mp4', salida, {
  onProgress: ({ bytesWritten, totalBytesExpectedToWrite }) => {
    setAvance(bytesWritten / totalBytesExpectedToWrite);
  },
});
await tarea.start();
```

También existe `archivo.watch(callback)`, que avisa cuando un archivo o directorio cambia. Útil si dos partes de la app tocan lo mismo.

Cerrar el círculo: guardar la foto

```ts
import { File, Directory, Paths } from 'expo-file-system';

async function guardarFoto(uriDeLaCamara: string) {
  const galeria = new Directory(Paths.document, 'fotos');
  if (!galeria.exists) galeria.create();

  const temporal = new File(uriDeLaCamara);            // está en la caché
  const definitiva = new File(galeria, `${Date.now()}.jpg`);

  temporal.copy(definitiva);                            // ahora sí sobrevive
  return definitiva.uri;
}
```

> `takePictureAsync` y `launchImageLibraryAsync` devuelven una URI en la **caché**. Si tu app guarda la referencia en una base de datos y no copia el archivo, en unos días la imagen no está más. Es un bug que aparece en la entrega final, nunca en la demo.

UNIDAD 4 · PARTE 5

Multimedia

Reproducir video, reproducir audio y grabar con el micrófono.

Dos paquetes, un mismo patrón

`expo-av` **ELIMINADO EN SDK 55**

Traía `<Video>` y `Audio.Sound` en un solo paquete. **Ya no existe**: instalarlo no es una opción, y casi todo el material que van a encontrar googleando lo usa.

`expo-video` + `expo-audio`

Se partió en dos. Los dos comparten la misma idea: un **reproductor** —el objeto que tiene el estado— y, si hay algo que mostrar, una **vista** aparte que lo dibuja.

> El reproductor **no vive en el estado de React**: cambiarle una propiedad no re-renderiza nada. Para mostrar en pantalla si está reproduciendo hay que **escuchar sus eventos** — es el mismo patrón que los sensores de la Parte 3.

Video

```tsx
import { useEvent } from 'expo';
import { useVideoPlayer, VideoView } from 'expo-video';

export function Reproductor({ fuente }: { fuente: string }) {
  const player = useVideoPlayer(fuente, p => { p.loop = true; p.play(); });

  // el estado NO es de React: hay que escucharlo
  const { isPlaying } = useEvent(player, 'playingChange',
                                 { isPlaying: player.playing });

  return (
    <>
      <VideoView style={e.video} player={player}
                 fullscreenOptions={{ enable: true }} allowsPictureInPicture />
      <Button title={isPlaying ? 'Pausar' : 'Reproducir'}
              onPress={() => isPlaying ? player.pause() : player.play()} />
    </>
  );
}
```

> Seguir reproduciendo con la app en segundo plano y el picture-in-picture **no se prenden desde el código**: van en el config plugin de `expo-video` en `app.json` y piden recompilar.

Audio: reproducir y grabar

REPRODUCIR

```
import { useAudioPlayer } from 'expo-audio';

const sonido = useAudioPlayer(require('./assets/ok.mp3'));

<Button title="Sonar" onPress={() => {
  sonido.seekTo(0);
  sonido.play();
}} />
```

GRABAR

```
import { useAudioRecorder, useAudioRecorderState,
         RecordingPresets, AudioModule } from 'expo-audio';

const grabador = useAudioRecorder(RecordingPresets.HIGH_QUALITY);
const estado   = useAudioRecorderState(grabador);

await AudioModule.requestRecordingPermissionsAsync();

await grabador.prepareToRecordAsync();
grabador.record();
// ...
await grabador.stop();     // el archivo queda en grabador.uri
```

Por defecto la grabación se guarda en la **caché** — el mismo problema que la foto de la cámara. Se resuelve igual: copiándola con `expo-file-system`, o pasándole `{ ...RecordingPresets.HIGH_QUALITY, directory: 'document' }`.

UNIDAD 4

Lo que queda

- — el deck que sigue: locales y push.
- — dónde se guarda lo que hoy escribimos en un archivo: SQLite, almacenamiento clave-valor y el servidor.

`docs.expo.dev/versions/latest/sdk/camera` · `/imagepicker` · `/location` · `/map-view` · `/sensors` · `/haptics` · `/filesystem` · `/video` · `/audio`